

Верификация архитектуры «СТО Challenge»

AI Архитектор

 REC

Проверить, идет ли запись

Меня хорошо видно
& слышно?





Андрей Носов

Ведущий AI-архитектор, Рафт

- 13 лет в разработке решений на основе NLP и GenAI
- PhD Communication Sciences, Tampere University
- Руководитель школы AI-архитекторов OTUS

 @parallelnominded

 andrey.nosoff@gmail.com



Правила вебинара



Активно
участвуем



Задаем вопрос
в чат или голосом



Вопросы вижу в чате,
могу ответить не сразу

Правила вебинара



Off-topic обсуждаем
в закрытом чате в Telegram

Кто еще не присоединился,
сканируйте QR код :)



Подготовка к СТО challenge

Что такое СТО Challenge и зачем готовиться?

Концепт: Это "Quality Gate" между проектированием и тратой бюджета. В нашем кейсе это аудит целесообразности инвестиций в размере **7.3 млн руб.**

✓ Definition of Ready

- ✓ **Проблема понятна**
Есть утвержденный PRD (Product Requirements Document) с бизнес-целями.
- ✓ **Решение визуализировано**
C4 Model (Context + Container diagram) показывает границы системы.
- ✓ **Цена посчитана**
Смета CAPEX (разработка) + Прогноз OPEX (эксплуатация).
- ✓ **Риски названы**
Таблица рисков с четким планом митигации (Mitigation Plan).

STOP FACTOR

🚫 Почему разворачивают

- 🚫 **Нет документации**
Отсутствие Pre-read материалов за 48 часов до встречи.
- 🚫 **Бюджетный разрыв**
Решение не бьется с бюджетом (пример: K8s без денег на DevOps инженера).
- 🚫 **Отсутствие расчетов**
Ответы в стиле «Я думаю, выдержит» вместо «Расчет показывает запас 30%».

Верификация vs Валидация (Матрица защиты)

Защищаем не технологии, а ценность. Матрица вопросов СТО.

Уровень	Вопрос (Что спрашивают)	Артефакт для ответа (Что показать)
VERIFICATION Инженерный	⚙️ «Соблюдены ли NFR?» «Есть ли бэкапы? Правильно ли выбран протокол?»	• Sizing Calculator (Excel расчеты) • C2 Diagram (Container view) • NFR Checklist (Non-functional reqs)
VALIDATION Бизнесовый	📈 «Решает ли это задачу?» «Окупится ли разработка? Не уйдет ли клиент из-за задержки?»	• ADR (Architecture Decision Record) • User Journey Map (Путь пользователя) • Business Constraints (Бизнес-ограничения)



Лайфхак защиты

Если СТО задает инженерный вопрос: «Почему Docker Swarm?»

Отвечайте через Валидацию: «Потому что для MVP на 1000 юзеров Swarm экономит нам 160 часов работы DevOps (300к руб), при этом обеспечивая нужный аптайм».

Пакет документов

«Без бумажки — нет архитектуры». Обязательный минимум.



High-Level Design (C4)

Контекст и Контейнеры

01

Показывает границы системы и внешние интеграции. Отвечает на вопрос: «С кем мы общаемся и как?»

- Контекст: GreenAPI, ERP, Jivo
- Контейнеры: API Gateway, Bot Core, DB

 PlantUML / Structurizr



Sequence Diagram

Для критических сценариев

02

Показывает поток данных и где теряется время (Latency). Критично для проверки SLA.

Кейс: *User → WhatsApp → Adapter → Queue → Orchestrator → LLM → ...*

 Mermaid / PlantUML



Infrastructure Bill

Sizing & Cost

03

Математическое обоснование железа под нагрузку. Не просто «возьмем сервер», а расчет.

- Расчет CPU/RAM/GPU под RPS
- Смета CAPEX + Прогноз OPEX

 Excel / Google Sheets



Risk Register

Реестр рисков

04

Честный список того, что может пойти не так, с планом действий (Mitigation Plan).

- Технические риски (ERP упадет)
- Продуктовые риски (LLM загрузит)

 Confluence / Markdown

Инструмент АТАМ — Utility Tree (Дерево полезности)

Формализация требований. Превращаем «Надежно» в Метрики.

Легенда приоритетов (Важность / Сложность):

Н/Н (High/High): Высокая важность / Высокая сложность. К этим пунктам готовимся на 100%.

Н/М (High/Medium): Важно, но проще реализовать.

Utility Tree



Performance

Ref: NFR-01

LATENCY

«Система отвечает на вопрос по базе знаний < 10 сек (P95)»

Н/Н



Availability

Ref: NFR-02

FAULT TOLERANCE

«При падении ERP бот продолжает отвечать (используя кэш или заглушку)»

Н/М



Data Quality

Ref: NFR-05

ACCURACY

«Бот не выдумывает факты, отсутствующие в Confluence (Hallucination rate < 5%)»

Н/Н

Trade-offs (Компромиссы)

Идеальных решений нет. Что мы принесли в жертву?

LLM Model Strategy

Выбор: Open-Source (Llama/Mistral) on-premise

+ ПРЕИМУЩЕСТВА (PROS)

- ✓ **Приватность данных:** Данные не покидают периметр (не шлем в OpenAI).
- ✓ **Cost Control:** Нет цены за токен (Pay-per-token), предсказуемый OPEX.

- НЕДОСТАТКИ (CONS)

- ✗ **CAPEX на железо:** Требуется мощный GPU сервер (T4/A10).
- ✗ **Качество:** Llama-70b уступает GPT-4 в сложных рассуждениях.

Caching Strategy

Выбор: Redis Cache (TTL 300s)

+ ПРЕИМУЩЕСТВА (PROS)

- ✓ **Снижение нагрузки:** Защищаем медленную ERP от спайков трафика.
- ✓ **Скорость:** Мгновенный ответ (<50ms) для частых запросов.

- НЕДОСТАТКИ (CONS)

- ✗ **Риск консистентности:** Риск показать устаревшую цену товара.
- ✗ **Complexity:** Нужно обрабатывать инвалидацию кэша (Cache Invalidation).

💡 **Методика выявления Trade-offs:** «Мы выбрали [Решение A], что дает [Плюс X], но ухудшает [Минус Y]».

Методика расчета: Производительность (Performance Sizing)

Ответ на вопрос «Выдержим ли 100k RPS?». Математическое обоснование.

📊 Формула 1: Закон Литтла (Little's Law)

$$L = \lambda \times W$$

РАСЧЕТ CONCURRENCY:

λ (RPS) = 100 req/sec (пиковая нагрузка)

W (Latency) = 5 sec (LLM отвечает долго)

$L = 100 \times 5 = 500$ одновременных запросов

⚠️ ВЫВОД

Нам нужно держать **500 потоков/воркеров**. Один синхронный Python-контейнер не вывезет. **Решение:** Нужен AsyncIO (FastAPI) + масштабирование реплик (Horizontal Scaling).

⌚ Формула 2: LLM Latency

$$T_{total} = T_{prefill} + (N_{tokens} \times T_{per_token})$$

РАСЧЕТ ВРЕМЕНИ ОТВЕТА:

T_{token} (на GPU T4) ≈ 25 ms

N (Длина ответа) = 200 токенов

$T_{gen} = 200 \times 0.025 = 5.0$ секунд

**Без учета префилла и сети*

💡 ИНСАЙТ

Физический предел генерации — 5 секунд. Ускорить кодом (оптимизацией Python) нельзя, только более мощным железом (A10/A100) или квантованием.

Методика расчета: Storage Sizing

Когда закончится место на диске? Анализ данных vs Логи.



1. Векторная база (Qdrant)

База знаний Confluence

Объем данных	1,000 страниц
Чанкинг (1 стр = 10)	10,000 векторов
Размер вектора (ada-002)	1536 float32 ≈ 6 КБ

ИТОГОВЫЙ ОБЪЕМ

~60 MB

RAM Usage (Server 16GB)

✓ Вывод

Влезает в RAM любой микроволновки. Шардирование не нужно. Вся база легко кэшируется в оперативной памяти для мгновенного поиска.



2. Логи (Observability)

ELK / Loki Storage

Нагрузка (RPS)	20 req/sec (avg)
Событий в сутки	20 * 86400 ≈ 1.7 млн
Размер записи лога	~2 КБ (JSON)

ОБЪЕМ ЗА МЕСЯЦ

~100 GB

Disk Usage (SSD 256GB)

⚠ Риск

Обычный диск 500GB забьется за 4-5 месяцев. **Решение:** Нужна политика ротации (Log Rotation 30 дней) или автоматическая выгрузка архивов в S3 (Cold Storage).

Бюджет и ТСО: сколько это стоит на самом деле?

CAPEX (РАЗОВЫЕ ЗАТРАТЫ)

7.3 млн ₽

Срок: 53 дня

Разработка (FTE)	65%
Менеджмент (PM/Arch)	20%
Риски и резерв	15%

*Согласно утвержденной смете MVP

OPEX (ЕЖЕМЕСЯЧНО)

~100 тыс ₽/мес

Инфраструктура и лицензии

GPU Server (T4/A10)	~\$500
Managed DB (PG + Redis)	~\$100
Traffic & Balancers	~\$50
GreenAPI License	Fix

Аргумент для защиты (TCO Analysis)

Разработка (CAPEX): 7.3 млн

Эксплуатация (1 год): ~1.2 млн



«Стоимость владения за год (1.2 млн) составляет **всего 15%** от стоимости разработки. Архитектура экономически эффективна: мы не тратим бюджет на проприетарные API (OpenAI), инвестируя в собственное железо.»

Управление рисками

Не скрывать, а управлять. Митигация ключевых угроз MVP.

ID	РИСК (THREAT SCENARIO)	ПЛАН МИТИГАЦИИ (MITIGATION PLAN)
 #1	Грязные данные в Confluence LLM будет галлюцинировать, если скормливать ей сырой HTML с таблицами, макросами и мусором. Риск низкого качества ответов.	 ТЕХНИЧЕСКОЕ РЕШЕНИЕ В архитектуру добавлен модуль препроцессинга (ETL Pipeline) и ручная валидация "Golden Set" (тестовая выборка из 100 пар вопрос-ответ).  Заложено 40 часов Data Scientist
 #2	Медленная ERP (Timeouts) ERP система заказчика периодически отвечает > 10 секунд. Бот будет "висеть" в ожидании ответа.	 ПАТТЕРН ПРОЕКТИРОВАНИЯ Использование паттернов "Cache-Aside" (кэширование цен) и "Circuit Breaker" (прерыватель). Бот не ходит в ERP на каждый чих, при сбое отдает последнее известное значение.
 #4	Зависимость от GreenAPI Vendor Lock-in. Если провайдер WhatsApp API закроется или изменит цены, проект встанет.	 АРХИТЕКТУРНАЯ ИЗОЛЯЦИЯ Выделенный слой адаптеров (Hexagonal Architecture / Ports & Adapters). Логика бота не знает о GreenAPI. Замена провайдера потребует переписать только 1 модуль адаптера.  Оценка замены: 32 часа
 Всего идентифицировано: 7 рисков  Критических (Showstoppers): 0 (все имеют Mitigation Plan)  Бюджет на риски: 15% от CAPEX		

Фиксация решений (ADR)

«Входной билет» на защиту. Проект решения, с которым вы приходите.

ADR-001

Использование Docker Swarm для MVP

PROPOSED (DRAFT)

CONTEXT (КОНТЕКСТ)

Команда DevOps ограничена (1 человек, частичная занятость). Требуется оркестрация 5-7 микросервисов. Ожидаемая нагрузка на MVP < 50 RPS. Жесткий дедлайн (53 дня).

ALTERNATIVES CONSIDERED (РАССМОТРЕНО)

Docker Swarm

Kubernetes (K8s)

Docker Compose (Bare Metal)

Serverless (AWS Lambda)

DECISION (РЕШЕНИЕ)

Мы будем использовать Docker Swarm.

Status: Accepted for MVP phase only.

RATIONALE (ОБОСНОВАНИЕ)

K8s требует setup time > 40 часов (в смете заложено всего 16). Docker Compose не обеспечивает Zero Downtime Deployment. Swarm предоставляет баланс: простой конфиг (похож на Compose), но есть оркестрация и rolling updates из коробки.

CONSEQUENCES (ПОСЛЕДСТВИЯ)

Budget Compliance

Укладываемся в бюджет (16 часов) и сроки.

Low Cognitive Load

Разработчикам не нужно учить Helm чарты.

Scaling Limitations

Сложнее скейлить stateful нагрузки по сравнению с K8s Operators.

Technical Debt

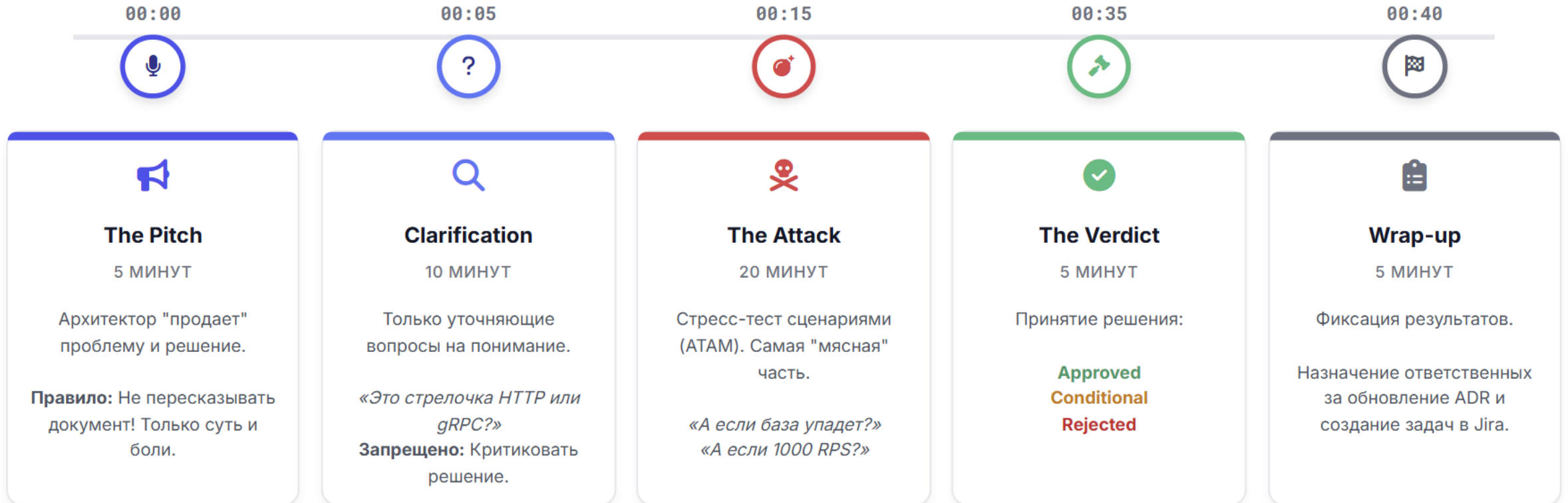
При росте > 500 RPS потребуются миграция на K8s (фаза 2).

Совет: Приходите на челлендж с готовым черновиком. Это показывает проработку альтернатив и экономит время встречи.



Регламент и структура сессии

Жесткий тайминг — залог выживания. Формат «30-Minute Architecture Check».



Проведение СТО challenge

Ролевая модель (Who is Who)

Участники процесса, их цели и инструменты.



The Defender
АРХИТЕКТОР

ГЛАВНАЯ ЗАДАЧА

Защитить TCO и жизнеспособность решения.

«Я знаю о рисках, вот план их митигации. Решение экономически оправдано.»



The Challenger
СТО / TECH LEAD

ГЛАВНАЯ ЗАДАЧА

Найти, где компания потеряет деньги или репутацию.

Инструмент: Вопросы «What-if» (А что если база упадет? А если нагрузка x10?)



SMEs
EXPERTS (OPS, SEC)

ФОКУС ВНИМАНИЯ

Ops/DevOps: Эксплуатация, логи, бэкапы.

Sec: Утечки данных, IDOR, Injection.



The Facilitator
МОДЕРАТОР

ГЛАВНАЯ ЗАДАЧА

Убивать оффтоп и следить за таймингом.

Полномочия: Может удалить участника, который не читал Pre-read документацию.

Фаза 1 — Идеальный Питч (The Hook)

Как продать архитектуру за 5 минут. Структура выступления.

1 CONTEXT (ЗАЧЕМ МЫ ЗДЕСЬ?)

Опишите бизнес-проблему и ценность. Почему проект вообще существует?

ПРИМЕР (RAG-БОТ):

«Мы строим RAG-бота, чтобы разгрузить первую линию техподдержки на 30% и сократить время ответа.»

2 CONSTRAINTS (ОГРАНИЧЕНИЯ)

Обозначьте рамки: бюджет, сроки, команда, легаси. Это оправдывает компромиссы.

ПРИМЕР:

«Денег мало (бюджет MVP), сроки жесткие (53 дня), команда DevOps неполная (part-time).»

3 KEY DECISION (ГЛАВНОЕ РЕШЕНИЕ)

Какую "ставку" вы сделали? Основная технология или подход.

ПРИМЕР:

«Главное решение — использовать готовую Open-Source LLM + Qdrant, чтобы не писать ML-модель с нуля.»

4 MAJOR RISKS (МЫ ЗНАЕМ О ПРОБЛЕМАХ)

Покажите, что вы контролируете ситуацию. Назовите риски первыми.

ПРИМЕР:

«Мы знаем, что GreenAPI ненадежен и данные в Confluence грязные. Для этого у нас есть адаптеры и ETL.»



— АНТИПАТТЕРН (КАК НЕ НАДО)

Начинать с деталей реализации: «Ну, мы взяли Python 3.11 и библиотеку Pydantic...» .
СТО это не интересно на старте. Ему важно понять, окупится ли проект и решает ли он задачу бизнеса.

Фаза 2 — Атака сценариями (АТАМ Attack)

Как «ломать» архитектуру правильно. Методика генерации сценариев отказа.



1. Ресурсный отказ

Отказ внутреннего компонента системы (База, Кэш, Диск).

ВОПРОС ЧЕЛЛЕНДЖЕРА:

«Упал Redis. Что видит юзер? 500-я ошибка, белый экран или просто старые данные из локального кэша?»



2. Сетевой отказ

Проблемы с внешними интеграциями и сетью (Timeout, Latency).

ВОПРОС ЧЕЛЛЕНДЖЕРА:

«GreenAPI отвечает 30 секунд (Timeout). Висят ли воркеры Python в ожидании? Забьется ли память процессами?»



3. Пиковая нагрузка

Резкий рост трафика, превышающий расчетный (Black Friday).

ВОПРОС ЧЕЛЛЕНДЖЕРА:

«Пришли 1000 юзеров одновременно. Встанет ли очередь запросов в Postgres? Какая максимальная глубина очереди?»



4. Эволюция системы

Проверка гибкости архитектуры к будущим изменениям.

ВОПРОС ЧЕЛЛЕНДЖЕРА:

«Через месяц бизнес захочет добавить Telegram. Придется переписывать Core логику или только адаптер?»



🔥 ЛАЙФХАК ДЛЯ СТО

Не верьте словам. Требуйте показать пальцем на диаграмме C2/C3:

«Покажи, в каком именно квадратике работает Retry, а где Circuit Breaker?»

Проверка масштабируемости

Математика против магии. Расчет «на салфетке» (Back-of-the-envelope).

СЦЕНАРИЙ ДИАЛОГА НА ЗАЩИТЕ

C

«Ты говоришь, пиковая нагрузка **100 RPS**. У нас синхронный Python (Flask/Django)?»

A

«Да, мы взяли стандартный стек для быстрого старта.»

C

«Смотри. Один запрос к LLM идет **5 секунд**. Значит, 1 процесс держит всего 0.2 RPS. Чтобы держать 100 RPS, нужно 500 процессов. Каждый ест 50 МБ RAM.
Итого 25 ГБ RAM только на воркеры. У нас сервер 16 ГБ. Система умрет.»

A

«... (Молчание. Архитектор не прошел проверку математикой).»

РАСЧЕТ НАГРУЗКИ (LITTLE'S LAW)

RPS (λ)

100 req/s

Latency (W)

5.0 sec

Concurrency (L)

$\lambda \times W = 500$

RAM per Worker

50 MB

REQUIRED RAM

25 GB

Available: 16 GB (Crash!)

⚠

Синхронная модель не подходит для IO-bound задач с высокой латентностью (LLM).

✓

Переход на AsyncIO (FastAPI/Aiohttp) — 1 процесс держит 1000+ соединений.

✓

Очередь задач (Celery/RabbitMQ) для тяжелых вычислений.

✓

Уменьшение лимитов (Rate Limiting) до емкости железа.

Проверка безопасности (STRIDE)

Где нас взломают? Методика моделирования угроз (упрощенная).

УГРОЗА (THREAT)	ВОПРОС ЧЕЛЛЕНДЖЕРА (ATTACK)	РЕШЕНИЕ В АРХИТЕКТУРЕ (DEFENSE)
 Spoofing ПОДМЕНА ЛИЧНОСТИ	«Могу ли я притвориться другим юзером, просто подменив номер телефона в JSON входящего вебхука?»	 AUTHENTICATION / INTEGRITY Проверка цифровой подписи провайдера для каждого запроса. HMAC-SHA256 JWT Verification
 Tampering ПОДДЕЛКА ДАННЫХ	«Могу ли я инъекцией в промпт заставить бота слить конфиденциальную статью из Confluence, которую мне нельзя видеть?»	 INPUT VALIDATION Слой очистки и валидации промптов перед отправкой в LLM. Sanitization Layer Prompt Guard
 DoS ОТКАЗ В ОБСЛУЖИВАНИИ	«Что если я отправлю 1000 сообщений в секунду? Бот ляжет для всех остальных клиентов?»	 AVAILABILITY PROTECTION Ограничение количества запросов на API Gateway и уровне приложения. Rate Limiting Token Bucket



Золотое правило защиты: На каждый пункт STRIDE должен быть ответ в архитектуре (диаграмме или ADR).
Если ответа нет — это уязвимость архитектуры.

Фаза 3 — Принятие решений

Светофор решений: критерии оценки архитектуры.

 APPROVED	 CONDITIONAL	 REJECTED
ЗНАЧЕНИЕ Можно кодить. КОГДА СТАВИТЬ ● Риски понятны и приняты (Accepted Risk). ● Бюджет сходится с планом. ● Нет критических блокировок.	ЗНАЧЕНИЕ «Да, но...» КОГДА СТАВИТЬ ● Требуются доработки (Add Backup, Clarify License). ● Кодить можно, но деплоить нельзя. ● Есть вопросы по Scalability.	ЗНАЧЕНИЕ Блокировка. КОГДА СТАВИТЬ ● Найден критический риск (SPOF, дыра в безопасности). ● Смета занижена в разы. ● Решение не решает задачу бизнеса.
STATUS: READY FOR DEV	ACTION: SCHEDULE FOLLOW-UP	ACTION: REDESIGN REQUIRED



Важно: Статус **"Conditional"** требует обязательного назначения даты повторного мини-ревью (Follow-up) для проверки устранения замечаний. Без даты это **"Rejected"**.

Психология защиты

Как отвечать на неудобные вопросы и не выглядеть глупо. Техники айкидо.



Признание (Ownership)

Не оправдывайтесь внешними обстоятельствами. Берите ответственность за компромиссы.

✖ ПЛОХО

«Ну, мне не дали времени, менеджер торопил...»

✔ ХОРОШО

«Да, это технический долг. Мы приняли его осознанно, чтобы успеть к дедлайну. Исправим в Q2.»



Уточнение контекста (Priorities)

Возвращайте дискуссию к целям бизнеса и Utility Tree, если спор ушел в технологии.

✖ ВОПРОС

«Почему база не в кластере?»

✔ ОТВЕТ

«Согласно Utility Tree, доступность 99.9% не является приоритетом для MVP. Репликация удваивает бюджет.»



Yes, and... (Да, и...)

Не спорьте с фактом риска. Согласитесь с ним и покажите план решения (Mitigation).

✖ ПЛОХО

«Нет, Postgres выдержит, он мощный!»

✔ ХОРОШО

«Да, Postgres может стать узким местом при росте x10, и поэтому мы заложили шардирование в ADR на будущее.»



«Хороший архитектор — пессимист на этапе дизайна и оптимист на этапе реализации. На Challenge будьте профессиональными пессимистами.»

Фиксация результатов (Artifact Update)

Если не записано — значит, этого не было. Чек-лист действий после встречи.



1. Обновление ADR



Финализация статуса архитектурного решения. Дописываем выявленные риски в раздел "Consequences".

Draft



Accepted

OR

Rejected



2. Корректировка Сметы



Добавляем забытые задачи, выявленные в ходе обсуждения (CAPEX/OPEX).

Пример: "Настройка бэкапов S3" – 4 часа



3. Задачи в Jira/Tracker



Создание технических задач на Mitigation Plan (снижение рисков).

Пример: "Нагрузочное тестирование GreenAPI"



Золотое правило Фасилитатора

Встреча не закрывается и протокол не рассылается, пока не назначен конкретный ответственный за обновление ADR.

NO OWNER = NO RESULT

Ресурсы и ссылки (Toolkit)

Что взять с собой. Инструментарий для защиты архитектуры.



1. Методики

FRAMEWORKS

ATAM 

Architecture Tradeoff Analysis Method — классика от SEI для поиска компромиссов.

STRIDE 

Модель угроз от Microsoft для Security ревью (Spoofing, Tampering, etc).



2. Калькуляторы

SIZING

Cloud Pricing 

Официальные калькуляторы AWS / Azure / Yandex Cloud для проверки OPEX.

LLM Token Calculator 

Tiktoken или OpenAI tokenizer для точного расчета стоимости RAG запросов.



3. Шаблоны

ARTIFACTS

ADR Template 

Шаблоны MADR или Y-Statements для фиксации архитектурных решений.

C4 Model Tools 

PlantUML или Structurizr для создания диаграмм (Context + Container).



*«Хороший архитектор — пессимист на этапе дизайна и оптимист на этапе реализации.
На Challenge будьте профессиональными пессимистами.»*



Практика СТО challenge

Кейс: Confluence Bot — вводные

Сеттинг ролевой игры. Артефакты на столе.

PROJECT SUMMARY

PRD-2023-RAG-01

ПРОДУКТ  RAG-Бот Integration: WhatsApp / Jivo	БЮДЖЕТ & СРОКИ  7.3 млн ₽  53 дня (Hard Deadline)	ОГРАНИЧЕНИЯ (SLA)  10-16 sec Latency High RPS: 5-10	ИНФРАСТРУКТУРА  On-Premise  Swarm  VM
--	---	---	---



THE DEFENSE

Architect & PM Team

✓ Цель: Уложиться в бюджет и срок.

✓ Решение: MVP на простых технологиях.

«Мы сделали MVP дешево и сердито. Это работает на моем ноутбуке, значит, полетит и в проде. Риски? Решим потом.»

VS

THE CHALLENGERS

CTO, Ops & Security Lead



Цель: Найти дыры в TCO и безопасности. ✗

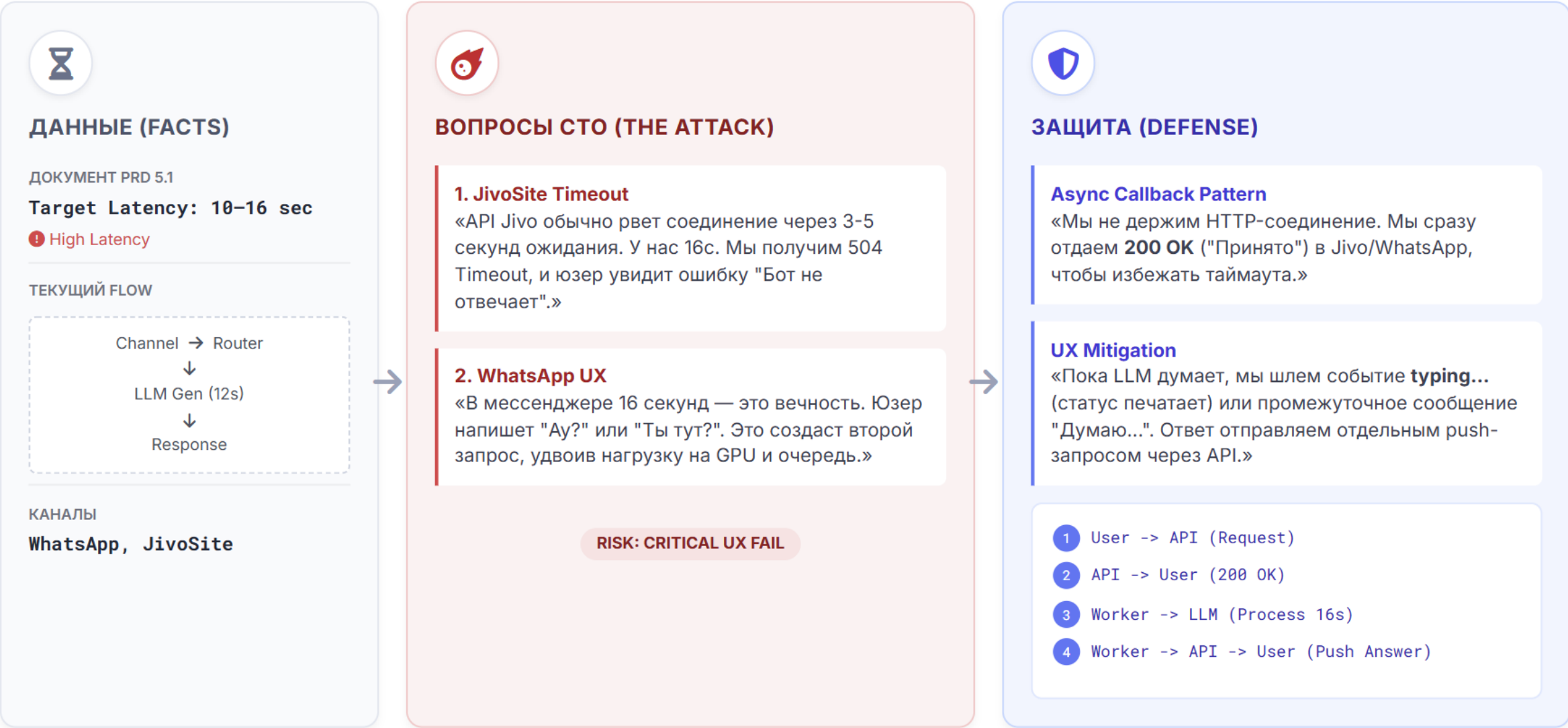
Инструмент: "What-if" сценарии. ✗

«Вы продали бизнесу бомбу замедленного действия. Без бэкапов и с латентностью 16 секунд пользователи уйдут в первый день.»

РАУНД 1

Атака на UX (Latency)

Выдержит ли пользователь 16 секунд тишины?



РАУНД 2

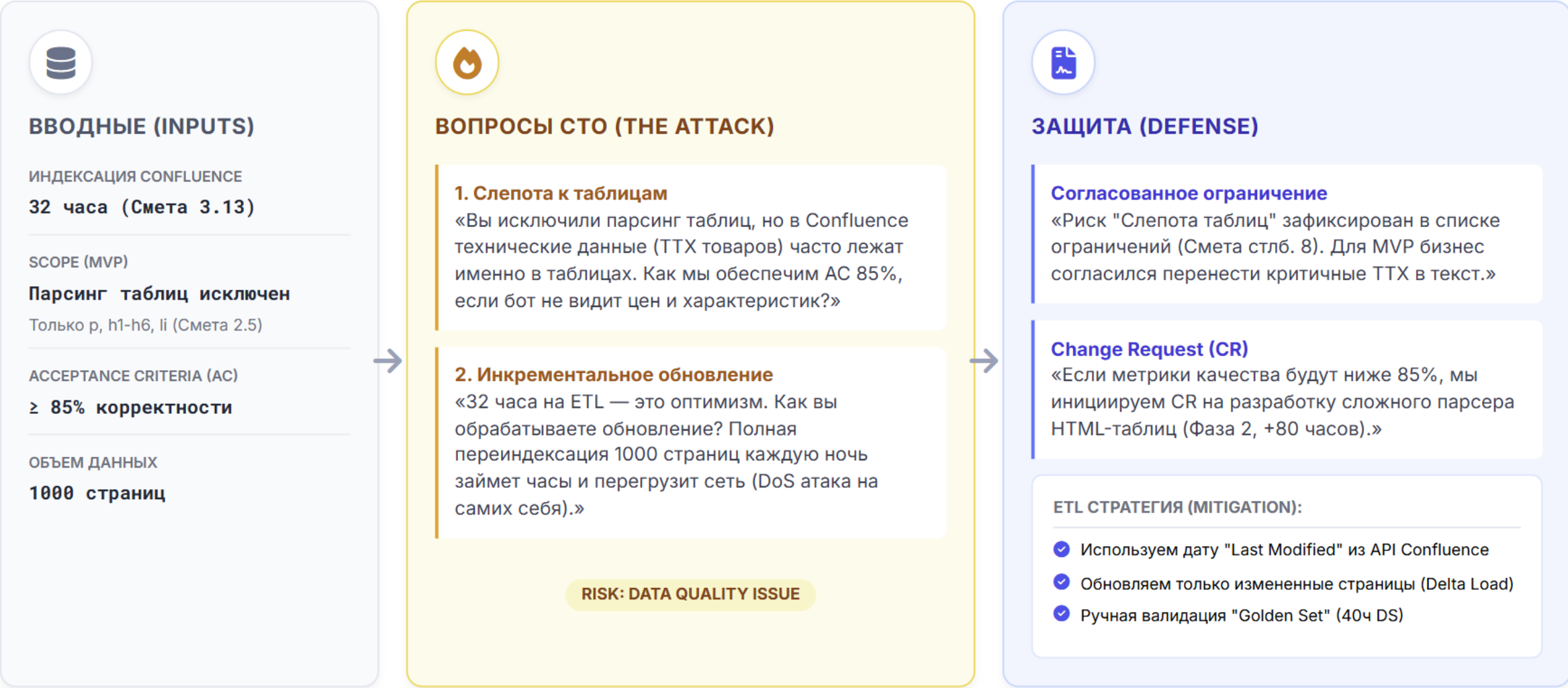
DevOps Gap

16 часов на инфраструктуру — оптимизм или халатность?



РАУНД 3 **Данные и ETL**

Мусор на входе = Галлюцинации на выходе



Раунд 4: Математика нагрузки

Little's Law в действии: выдержит ли Python 16 секунд ожидания?

СЦЕНАРИЙ ДИАЛОГА НА ЗАЩИТЕ

С

«Нам нужно держать 160 одновременных соединений (Concurrent Requests). У нас синхронный Python?»

«Да, стандартный Flask.»

А

С

«Считаем. 160 запросов = 160 процессов. Один процесс Python ~50 МБ. **Итого 8 ГБ RAM только на воркеры + ОС + БД.** У нас VM на 8 ГБ. Мы уйдем в OOM Killer при первом же пике.»

«Понял. Нужно менять стек на асинхронный.»

А

РАСЧЕТ НАГРУЗКИ (LITTLE'S LAW)

RPS (λ) 10 req/s

Latency (W) 16.0 sec

Concurrency (L) $\lambda \times W = 160$

RAM per Worker 50 MB

REQUIRED RAM (WORKERS ONLY)

8 GB

Total VM Capacity: 8 GB (Risk!)



Action Item: Обновить ADR

Синхронный подход математически нежизнеспособен.

- ✓ **Обязательно:** AsyncIO стек (FastAPI / Aiohttp).
- ✓ **Закрепить в ADR:** "Decision: Use FastAPI due to high concurrency requirement".
- ✓ **Риск:** Проверить квалификацию команды (знают ли async?).

SECURITY

Атака на безопасность: HMAC, IDOR, Spoofing

Где нас взломают? Уязвимости внешних интеграций.



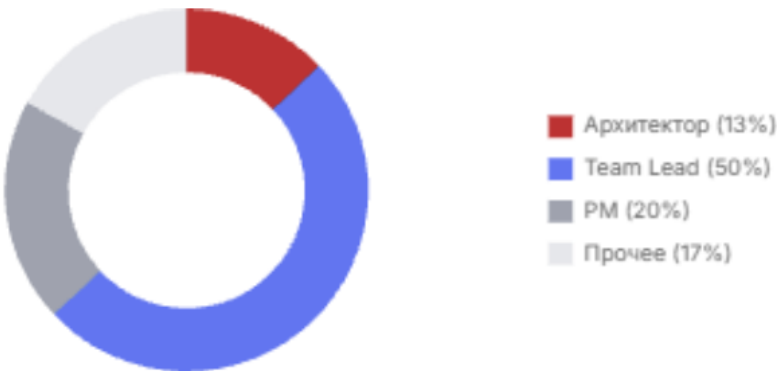
👥 Анализ трудозатрат и управления

Кто реально управляет хаосом? Проблема "Архитектора выходного дня".

Ресурсный план (Смета Гр. 0)

Архитектор (Arch)	56 часов (13% FTE)
Team Lead (TL)	212 часов (50% FTE)
Project Manager (PM)	80 часов (20% FTE)

Распределение времени управления



Risk: Architectural Drift

Код "уедет" от схемы C4 через 2 недели без надзора.

🔍 Вопросы CTO (The Gap)

- ❓ **Кто обновляет документацию?**
«Архитектор работает 1 день в неделю. Когда API Jivo изменится во вторник, кто обновит Sequence Diagram? Разработчики этого не сделают.»
- ❓ **Кто проверит NFR?**
«Кто реально зайдет в код и проверит, что Rate Limiting реализован через Redis, а не через `time.sleep(1)`? У PM нет квалификации, Архитектора нет на месте.»

🛡️ Решение (Mitigation)

- ✅ **Делегирование TL (Team Lead)**
«Тимлид имеет 50% загрузки (Стр 0.5). Он берет на себя операционный контроль кода (Code Review) и соблюдение NFR.»
- ✅ **Роль Архитектора**
«Архитектор выполняет только верхнеуровневый надзор (Design Review) и ведение ADR. Он не смотрит каждый Pull Request, но утверждает ключевые изменения.»



Update Roles Matrix in Project Charter

📌 Матрица вердиктов по кейсу

Финальная оценка и условия допуска к разработке.

ОБЛАСТЬ (AREA)	ОЦЕНКА	КОММЕНТАРИЙ / ACTION ITEM	ВЛИЯНИЕ НА СМЕТУ
UX / Latency Пользовательский опыт	⚠️ WARNING	Принято 16 сек, но нужен Async Callback. Риск потери юзеров WhatsApp митигирован индикатором "typing..."	0 ч (Учтено в 3.16)
DevOps & Infra Надежность	❌ CRITICAL	Нет бэкапов. 16ч на Swarm — нереально. Action: Добавить задачи на Setup + Backup + Recovery Test.	+32 ч (New Tasks)
Data Quality Точность RAG	⚠️ WARNING	Риск игнорирования таблиц принят бизнесом. Ограничение зафиксировано в ADR-005. Mitigation: ручной перенос TTX.	0 ч (Accepted Risk)
Security Безопасность	❌ CRITICAL	Нет валидации HMAC GreenAPI. Action: Реализовать проверку подписи webhook.	+4 ч (Update 3.6)

ИТОГОВОЕ РЕШЕНИЕ (FINAL VERDICT)

📌 Conditional Approval

🔒 Условия:

«Мы даем добро на старт разработки (Development Phase), но **блокируем деплой в Production** до устранения красных пунктов (DevOps Backup, Security HMAC).
Повторное ревью через 2 недели.»

Total Budget Impact: +36 hours (~5%)

ADR Пример (Домашнее задание)

Как правильно зафиксировать "боль" в документе.

ADR-005: Отказ от парсинга таблиц Confluence в MVP

APPROVED

CONTEXT & PROBLEM

Context: Confluence страницы содержат сложные макросы, вложенные таблицы и images. RAG-бот должен отвечать по ТТХ товаров.

Constraint: Разработка универсального HTML-парсера таблиц требует R&D (непредсказуемо по времени), а дедлайн MVP жесткий — 53 дня.

CONSIDERED OPTIONS

☐ 1. Писать кастомный парсер (BeautifulSoup + Pandas) — риск срыва сроков.

☐ 2. Использовать платный SaaS (Unstructured.io) — выход за бюджет OPEX.

☒ 3. Парсить только простой текст (p, h1-h6, li) — дешево и быстро.

DECISION

Мы исключаем таблицы и картинки из индексации MVP.

Парсер будет извлекать только текстовые ноды: p, h1-h6, li, blockquote.

CONSEQUENCES

+ **Positive:** Гарантированно укладываемся в срок 53 дня. Экономим 80 часов разработки.

- **Negative:** Ответы по ТТХ могут быть неполными (Risk #1).

Mitigation Plan

Критичные ТТХ переносит заказчик (Content Manager) в текстовый вид или в поля ERP.

Этот документ является "щитом" архитектора: если качество ответов будет низким, вы ссылаетесь на ADR-005.

Верификация архитектуры и CTO Challenge • Слайд 29

Чему мы научились сегодня?

4 главных инсайта CTO Challenge.

01



Смета = Архитектура

Если задачи нет в Excel (CAPEX), её не будет и в коде.

Пример: Забыли задачу "Бэкапы" → получили риск потери данных в проде.

02



Latency убивает интеграции

Медленный RAG (16 сек) требует смены парадигмы с синхронной на асинхронную. **Вывод:** Проверяйте SLA внешних систем (Jivo timeout 5s) до написания кода.

03



Ограничения — это щит

Фраза "Это не входит в MVP согласно Смете (стр 2.5)" — лучшая защита от Scope Creep. **Совет:** Фиксируйте "Out of Scope" так же четко, как и требования.

04



Валидация > Верификация

На защите мы обсуждаем не "как работает Docker", а "как мы не потеряем деньги при падении сервера". **Focus:** Business Value & Risk Mitigation.



ВАШ СЛЕДУЮЩИЙ ШАГ

Начните с малого, чтобы победить хаос.

 Оформить 3 ключевых ADR сегодня

Список литературы

- Bass, L., Clements, P., & Kazman, R. (2012). *Software Architecture in Practice* (3rd ed.). Addison-Wesley. SEI Series in Software Engineering. [Pearson](#) | [SEI](#).
- Richards, M., & Ford, N. (2020). *Fundamentals of Software Architecture: An Engineering Rather Than a Design Approach*. O'Reilly Media. [O'Reilly](#) | [Amazon](#).
- Boehm, B. W. (1984). Verifying and Validating Software Requirements and Design Specifications. *IEEE Software*, 1(1), 75–88. [IEEE Xplore](#).
- Nygard, M. T. (2018). *Release It!: Design and Deploy Production-Ready Software* (2nd ed.). Pragmatic Bookshelf. [Pragmatic](#) | [GitHub repo](#).
- Google SRE Team. (2016). *Site Reliability Engineering: How Google Runs Production Systems*. O'Reilly Media. [Google SRE Book](#) | [Free PDF](#).
- Fowler, M. (2002). *Patterns of Enterprise Application Architecture*. Addison-Wesley. [Martin Fowler](#) | [P2P](#).
- Brown, S. (2015). *Software Architecture for Developers* (Vol. 1: A Practical Guide to C4 Model). [C4 Model](#) | [LeanPub](#).
- Nygard, M., & Henderson, J. P. (2011–2026). Architecture Decision Records (ADR). [adr.github.io](#) | [Original GitHub](#).
- Shostack, A. (2014). *Threat Modeling: Designing for Security*. Wiley. [Wiley](#) | [Adam Shostack](#).
- OWASP Foundation. (2025). *OWASP Top 10 for Large Language Model Applications*. [OWASP LLM Top 10](#) | [GitHub](#).
- Weinberg, G. M. (1985). *The Secrets of Consulting: Techniques, Tactics and Truths*. Dorset House. [Dorset House](#) | [PDF excerpts](#).
- Hohpe, G. (2020). *The Software Architect Elevator*. O'Reilly Media. [O'Reilly](#) | [Author Site](#).

Онлайн-инструменты

- AWS Pricing Calculator: [cal](#)Tiktokerizer (OpenAI-compatible): [tiktokenizer.vercel.app](#) — для расчета токенов и стоимости [culator.aws](#) — для оценки OPEX и TCO инфраструктуры.
- Little's Law Explanation: [Little's Law в IT](#) | [Performance Theory](#).